Rule Engine

Developer Reference

Enums · Strategies · Factory · Operators

`com.tt.nc.enums` · `com.tt.nc.strategy`

Table of Contents

1. **ConditionGroup Enum** — Formatters & separators for rule values
2. **ConditionAttribute Enum** — Registry of all rule attributes & their groups
3. **RuleDataTypeEnum** — DB code → logical data type mapping
4. **RuleEngineOperatorEnum** — Operator codes & expression strings
5. **RuleEngineConditionStrategy** — Interface & implementations overview
6. **RuleEngineListStrategy** — LIST / SCALAR expression builder
7. **RuleEngineDateStrategy** — DATE expression builder
8. **RuleEngineSingleValStrategy** — Single-value expression builder
9. **RuleEngineConditionFactoryHandler** — Factory: selects the right strategy
10. **ScalarAttribute** — Reflection-based SCALAR / COLLECTION classifier

---

1 ConditionGroup

ENUM

Defines how a rule attribute's stored values are joined/formatted before being handed to the expression-building strategies.

Constants

Constant	Behaviour
YES_NO	Returns values.get(0) — first value only
COMMA_SEPARATED	Joins all values with ","

COMMA_SEPARATED_SPECIAL_CASE	Same as COMMA_SEPARATED — apply extra normalisation as needed
DATES	Single: as-is | Pair: "d1 - d2" | Multiple: "d1 - d2, d3 - d4, …"
HYPHEN	Joins all values with "-"

Key Method: format(List<String>)

Returns null for empty/null input. Otherwise delegates to the enum-specific formatter lambda.

Separator Quick-Reference

ConditionGroup	Separator
COMMA_SEPARATED / _SPECIAL_CASE	,
HYPHEN	-
DATES	-
YES_NO	null (no joining)

Usage Example

```
// Get separator from a ConditionAttribute
String sep = ConditionAttribute.AIRLINE_CODE
                 .getGroup()
                 .getSeparator();   // ","

// Format a list of values
List<String> vals = List.of("EK", "FZ");
String result = ConditionGroup.COMMA_SEPARATED.format(vals); // "EK,FZ"
```

---

2 ConditionAttribute

ENUM

Central registry mapping every rule attribute to a unique integer ID and a ConditionGroup. Used by the rule-parsing pipeline to decide how to interpret stored criteria values.

Fields

Field	Description
id (int)	Stable identifier — must match DB / UI mapping
group (ConditionGroup)	Parsing strategy for the attribute's values

Complete Attribute Table

ID	Attribute	ConditionGroup
1	AIRLINE_CODE	COMMA_SEPARATED
2	AIRLINE_NAME	COMMA_SEPARATED
3	AIRLINE_TYPE	COMMA_SEPARATED
4	BASE_FARE	HYPHEN
5	BLACKOUT_INBOUND_DATE	DATES
6	BLACKOUT_OUTBOUND_DATE	DATES
7	BOOKING_END_DATE	DATES
8	BOOKING_START_DATE	DATES
9	CABIN_CLASS	COMMA_SEPARATED
10	CORPORATE_ID	COMMA_SEPARATED
11	CROSS_SELLING_PATH	YES_NO
12	CUSTOMER_TYPE	COMMA_SEPARATED
13	DEAL_CODE	COMMA_SEPARATED
14	DESTINATION_AIRPORT	COMMA_SEPARATED
15	DESTINATION_CITY	COMMA_SEPARATED_SPECIAL_CASE
16	DESTINATION_COUNTRY	COMMA_SEPARATED_SPECIAL_CASE
17	EXCLUDE_MULTI_CARRIER	YES_NO
18	FLIGHT_NO	COMMA_SEPARATED
19	FOR_EX	YES_NO
20	NATIONALITY	COMMA_SEPARATED_SPECIAL_CASE
21	IMPORT_PNR	YES_NO
22	IT_FARES	YES_NO
23	NO_OF_DAYS	COMMA_SEPARATED
24	NO_OF_PAX_ADULT	COMMA_SEPARATED

25	NO_OF_PAX_CHILD	COMMA_SEPARATED
26	NO_OF_PAX_INFANT	COMMA_SEPARATED
27	ORIGIN_AIRPORT	COMMA_SEPARATED
28	ORIGIN_CITY	COMMA_SEPARATED_SPECIAL_CASE
29	ORIGIN_COUNTRY	COMMA_SEPARATED_SPECIAL_CASE
30	PCC	COMMA_SEPARATED
31	RBD	COMMA_SEPARATED
32	SUPPLIER	COMMA_SEPARATED
33	TRAVEL_END_DATE	DATES
34	TRAVEL_START_DATE	DATES
35	TRIP_TYPE	COMMA_SEPARATED
36	VALID_DAYS	COMMA_SEPARATED
37	JOURNEY_TYPE	COMMA_SEPARATED
38	BRANCH_BOOKING	YES_NO
39	FARE_BASIS_CODES	COMMA_SEPARATED
40	FLIGHT_TYPE	COMMA_SEPARATED

Lookup by ID

```
// Safe lookup — returns Optional
Optional<ConditionAttribute> attr = ConditionAttribute.fromId(32);
// → Optional.of(SUPPLIER)

// Chaining to get ConditionGroup
ConditionGroup group = ConditionAttribute.fromId(1)
    .map(ConditionAttribute::getGroup)
    .orElseThrow();
// → COMMA_SEPARATED
```

---

3 RuleDataTypeEnum

ENUM

Abstracts DB-stored numeric type codes into logical data types consumed by the strategy layer.

Code → Type Mapping

DB Code(s)	RuleDataTypeEnum	Notes
1	INTEGER	Whole numbers
3	DATE	Date attributes
6	DOUBLE	Decimal numbers
7	LIST_OF_INTEGER	Collection of integers
8	STRING	Textual scalars
2, 4, 5	LIST_OF_STRING	2=manual CSV 4=dropdown 5=enum dropdown

> ℹ A static reverse-lookup Map (LOOKUP) is built at class-load time for O(1) fromCode() access.

API

```
RuleDataTypeEnum.fromCode(1);  // → INTEGER
RuleDataTypeEnum.fromCode(5);  // → LIST_OF_STRING
RuleDataTypeEnum.contains(99); // → false
// Invalid code → throws IllegalArgumentException("Unknown operator: 99")
```

---

4 RuleEngineOperatorEnum

ENUM

Maps operator codes (from DB/UI) to Drools-compatible operator strings used when generating rule expressions.

Operator Table

Code	Enum Constant	Drools String
0	IN	in
1	NOT_IN	not in
2	EQUAL	==
3	NOT_EQUAL	!=
4	LESS_THAN	<

5	GREATER_THAN	>
6	LESS_THAN_OR_EQUAL	<=
7	GREATER_THAN_OR_EQUAL	>=
8	MATCHES	matches
9	NOT_MATCHES	not matches
10	BETWEEN	not contains (range guard)

Lookup

```
RuleEngineOperatorEnum op = RuleEngineOperatorEnum.fromCode(5);
op.getOperator(); // ">"
// Unknown code → throws IllegalArgumentException
```

---

5 RuleEngineConditionStrategy

INTERFACE

Strategy interface — each implementation generates a Drools-compatible condition expression for a specific category of attribute (date, list, scalar).

Contract

```
String buildConditionExpression(
    RuleCriteriaElements   ruleCriteria,
    RuleEngineAttributeDTO ruleEngineAttributeDTO
) throws Exception;
```

Implementations

Bean Name	Class
ruleEngineDateExpression	RuleEngineDateStrategy
ruleEngineListExpression	RuleEngineListStrategy
ruleEngineIntegerExpression	RuleEngineSingleValStrategy

---

6 RuleEngineListStrategy

SERVICE · ruleEngineListExpression

Handles LIST_OF_STRING and LIST_OF_INTEGER attributes. Generates OR (IN) or AND (NOT_IN) compound expressions, using contains() for collection fields and == / != for scalar fields.

Core Decision Logic

Runtime Type	Operator	Output Pattern
Collection	IN	(f.contains(A) || f.contains(B))
Collection	NOT_IN	(!f.contains(A) && !f.contains(B))
Scalar	IN	(f == A || f == B)
Scalar	NOT_IN	(f != A && f != B)

Examples

Collection field — IN

```
// flightNo = ["MS123", "QR404"]  (runtime collection)
// Rule values: MS123, EK202
→  (flightNo.contains("MS123") || flightNo.contains("EK202"))
```

Scalar field — NOT_IN

```
// supplier = 1  (runtime scalar)
// Rule values: 1, 2
→  (supplier != 1 && supplier != 2)
```

> ℹ ScalarAttribute.isCollection(name) determines the runtime type at build time using reflection-cached sets.

---

7 RuleEngineDateStrategy

SERVICE · ruleEngineDateExpression

Generates Drools date comparisons. Handles EQUAL, GREATER_THAN, LESS_THAN, and BETWEEN. Applies special expansion logic for blackout date attributes.

Date Format

Direction	Format
Input	MM-dd-yyyy
Output	dd-MMM-yyyy HH:mm (e.g. 15-Jan-2025 00:00)

BETWEEN Boundary Rules

- Start date: inclusive (>=)
- End date: exclusive (<) — incremented by +1 day internally

Examples

GREATER_THAN

```
( departureDate > "15-Jan-2025 00:00" )
```

BETWEEN — Date Range

```
( departureDate >= "01-Jan-2025 00:00" ) &&
( departureDate <  "11-Jan-2025 00:00" )
```

Blackout Dates — Expanded

```
// Input range "01-01-2025 - 01-03-2025" expands to 3 days:
blackoutDates not contains("2025-01-01") &&
blackoutDates not contains("2025-01-02") &&
blackoutDates not contains("2025-01-03")
```

---

8 RuleEngineSingleValStrategy

SERVICE · ruleEngineIntegerExpression

Handles scalar attributes (STRING, INTEGER, DOUBLE). Supports simple comparison operators and BETWEEN for numeric types.

Value Formatting

Data Type	How Value Is Emitted

INTEGER / DOUBLE	Raw (no quotes) e.g. 1000
STRING	Double-quoted e.g. "USD"

Examples

Numeric GREATER_THAN

```
// price > 1000
(price > 1000)
```

String EQUAL

```
// currency == "USD"
(currency == "USD")
```

BETWEEN (numeric only)

```
// Input value: "1000 - 2000"
(price >= 1000) && (price <= 2000)

// Invalid format → IllegalArgumentException:
//  "Invalid range criteria value for attribute <name>.
//   Expected format: fromValue - toValue"
```

---

9 RuleEngineConditionFactoryHandler

SERVICE · Factory + Strategy

Central entry point for expression generation. Selects the correct RuleEngineConditionStrategy based on RuleDataTypeEnum and delegates to it.

Selection Logic

RuleDataTypeEnum	Strategy Selected
DATE	RuleEngineDateStrategy
LIST_OF_STRING | LIST_OF_INTEGER	RuleEngineListStrategy
STRING | INTEGER | DOUBLE	RuleEngineSingleValStrategy

Extending the Factory

- Create a new class implementing RuleEngineConditionStrategy
- Annotate with @Service("yourBeanName")
- Add a new RuleDataTypeEnum constant (if needed)
- Wire the new type → strategy mapping inside the factory

> ✅ The factory is stateless and thread-safe — it only holds injected Spring beans.

---

10 ScalarAttribute

UTILITY ENUM

Classifies RulesFlightBean fields as SCALAR or COLLECTION at class-load time using Java reflection. Used by RuleEngineListStrategy to select the right expression pattern.

Classification Rules

Field Runtime Type	Classification
Assignable from java.util.Collection	COLLECTION → uses contains()
Anything else	SCALAR → uses == / !=

Performance

- Reflection runs once in a static initializer block
- Results cached in SCALAR_FIELDS and COLLECTION_FIELDS (HashSet)
- Runtime lookups are O(1)

API

```
ScalarAttribute.isScalar("supplier")    // → true
ScalarAttribute.isCollection("flightNo") // → true

// Usage inside strategy:
if (ScalarAttribute.isCollection(attributeName)) {
    // build contains() expression
} else {
    // build == / != expression
```

```
}
```

> Renaming a field in RulesFlightBean will automatically update the classification after redeployment — no manual enum changes needed.
